

VITyarthi
Build Your Own Project

PROJECT REPORT

Contact Book

A command-line contact manager in Python with file storage

Student Name	[Your Name]
Registration Number	[Your Registration No.]
Course Name and Code	[Course Name and Code]
Faculty	[Faculty Name]
Submission Date	September 2026

1. Introduction

This project is a Contact Book that runs in the terminal. It stores a name, phone number, email address and postal address for each person and lets the user add, view, edit, delete, list and search these entries. The program is written in Python and uses only the standard library.

The main idea was to build something small but complete: a program with a clear menu, checks on the data the user types in, and storage that survives after the program is closed. The project applies the basics of Python taught in the course, namely functions, dictionaries, conditionals, loops, string handling, file handling and exception handling.

2. Problem Statement

Phone numbers and email addresses are often kept in different places such as notebooks, messages or simply remembered. This makes it slow to find a contact, and mistakes like a wrong number or a repeated entry are easy to make.

The problem is to build a simple contact manager that can run on any computer with Python, accepts only sensible data, and keeps the contacts saved between sessions. The objectives are:

- Provide add, view, edit, delete, list and search operations through a simple menu.
- Check phone numbers, email addresses and names before saving them.
- Save the contacts to a file and load them again when the program starts.
- Show clear messages when the user enters something wrong or the file has a problem.

3. Functional Requirements

The program is divided into three major functional modules. Each module is made up of one or more functions in `Contact_Book.py`.

ID	Module	Requirement
FR1	Contact Management	Add a new contact with name, phone, email and address. A duplicate or empty name is rejected.
FR2	Contact Management	View the full details of one contact by name.
FR3	Contact Management	Edit a contact. Fields left blank keep their old value.
FR4	Contact Management	Delete a contact by name.
FR5	Search and Reporting	List all contacts in alphabetical order.
FR6	Search and Reporting	Search contacts by full or partial name, ignoring upper and lower case.
FR7	Storage and Validation	Load contacts from <code>contacts.json</code> when the program starts and save after every change.
FR8	Storage and Validation	Validate phone (exactly 10 digits) and email (contains "@" and a dot after it) before saving.

3.1 Input and Output Structure

Operation	Input from user	Output shown
Add	Name, phone, email (optional), address (optional)	Success message or an error message
View	Name	Name, phone, email and address, or "Contact not found!"
Edit	Name, then new phone, email, address (blank = keep)	Updated message or an error message
Delete	Name	Deleted message or "Contact not found!"
List all	None	All contacts sorted by name, or "No contacts available."
Search	Part of a name	Matching names with phone numbers, or "No matching contacts found."

3.2 User Workflow

1. The program starts and loads any saved contacts from contacts.json.
2. The menu is displayed and the user types a number from 1 to 7.
3. The program asks for the details needed for that option, one by one.
4. The program checks the input, updates the data, saves the file if something changed, and prints a message.
5. The menu is shown again until the user chooses 7 to exit.

4. Non-Functional Requirements

Requirement	What it means here	How it is met
Usability	A first-time user should be able to use it without instructions.	Numbered menu, a prompt for every input, and short messages after every action.
Reliability	Contacts must not be lost when the program closes.	Data is written to contacts.json after every add, edit and delete.
Error handling	Wrong input or a bad file should not crash the program.	Invalid input is rejected with a message. File reading and writing use try/except.
Data validation and integrity	Only sensible data should be stored.	Empty names, duplicate names, bad phone numbers and bad emails are rejected.
Maintainability	The code should be easy to read and change.	One function per task, meaningful names and short comments. Validation is in separate small functions.

Requirement	What it means here	How it is met
Resource efficiency	Should run on any basic computer.	Uses only built-in modules and a small JSON file. No external libraries or internet needed.

5. System Architecture

The program follows a simple layered design. The user talks only to the menu. The menu calls the operation functions, which use the validation functions to check input and the in-memory dictionary to hold the contacts. The storage functions copy that dictionary to and from a JSON file.

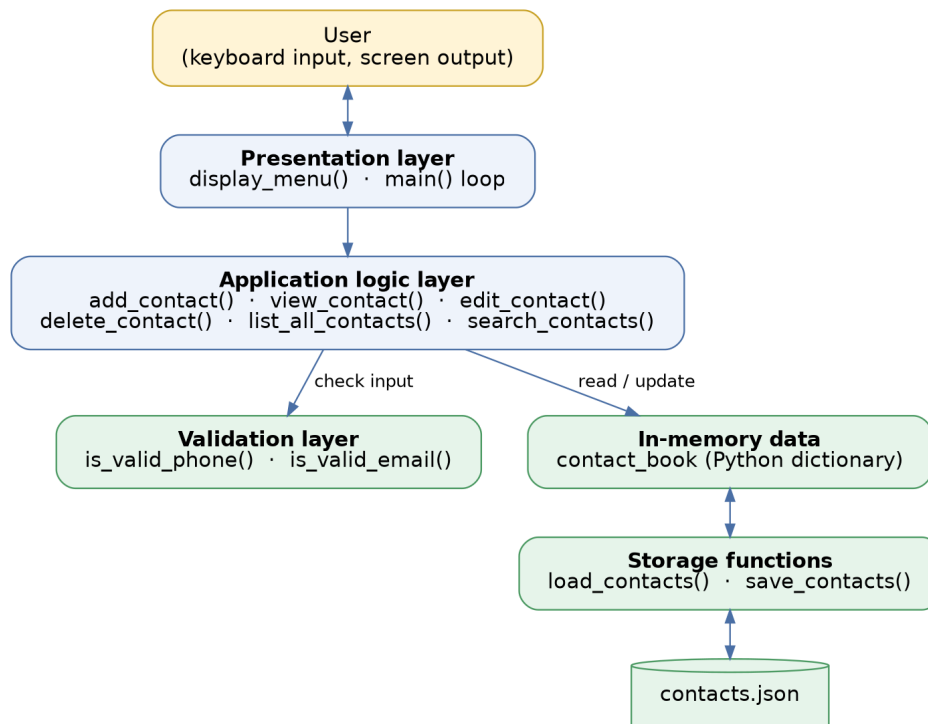


Figure 1: System architecture of the Contact Book

Presentation layer: display_menu() and the main() loop show the options and read the choice.

Application logic layer: six functions, one for each operation.

Validation layer: is_valid_phone() and is_valid_email() return True or False.

Data layer: the contact_book dictionary is the working copy, and load_contacts() and save_contacts() keep it in sync with contacts.json.

6. Design Diagrams

6.1 Use Case Diagram

There is one actor, the User. Adding and editing include validation, and adding, editing and deleting include saving the file.

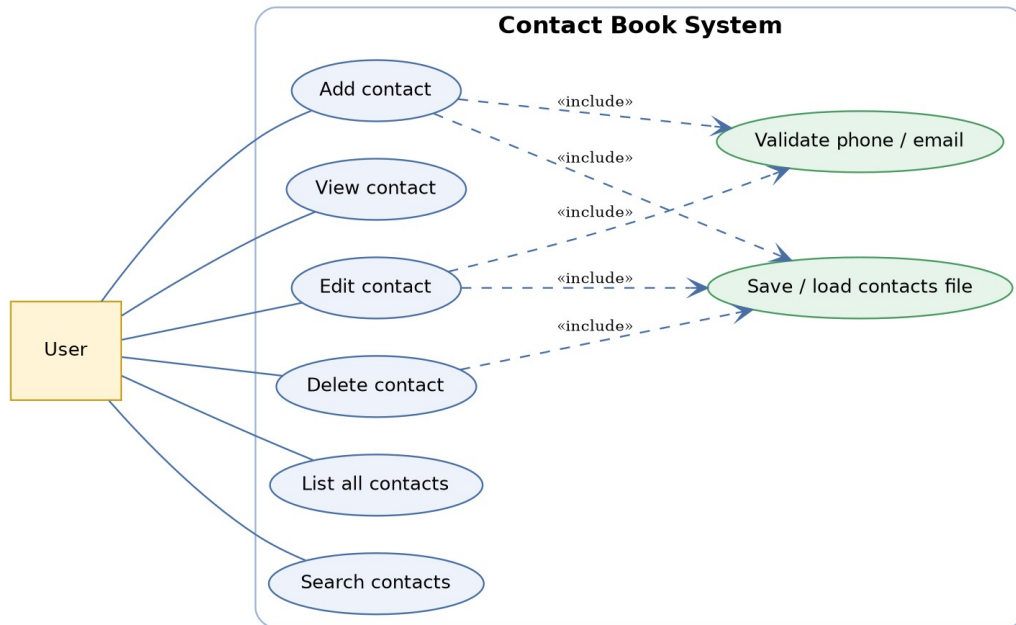


Figure 2: Use case diagram

6.2 Workflow Diagram

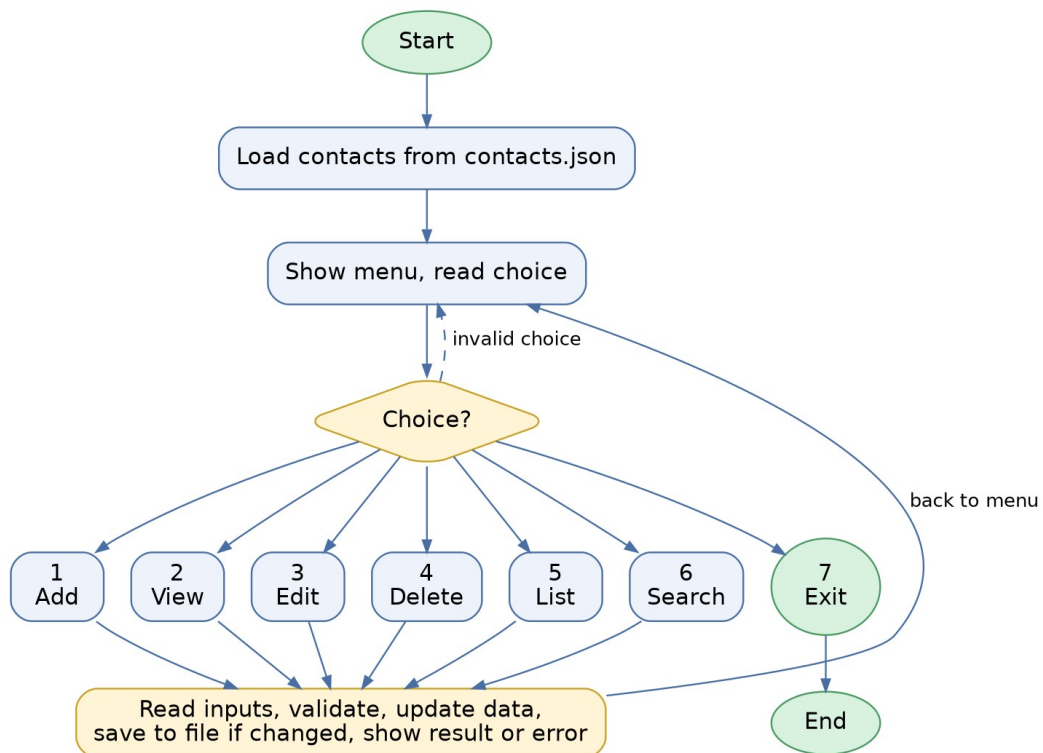


Figure 3: Workflow of the program

6.3 Sequence Diagram

The sequence below shows the steps for adding a contact, which is the operation that touches the most parts of the program.

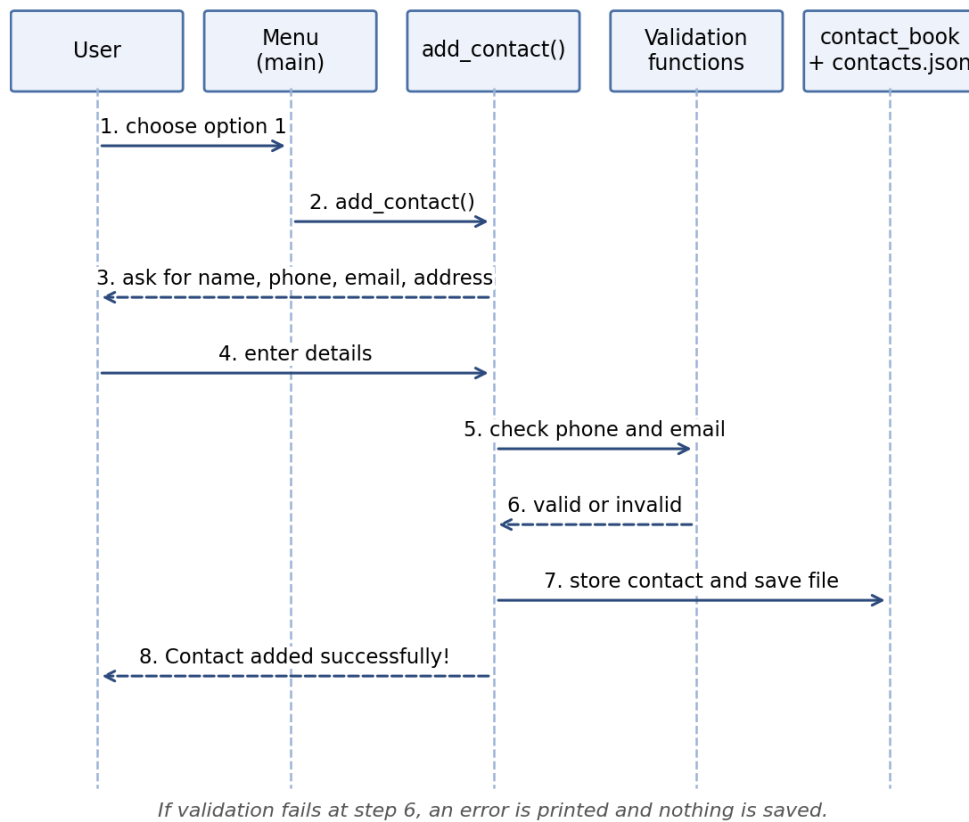


Figure 4: Sequence diagram for adding a contact

6.4 Component Diagram

The program is written with functions instead of classes, so a component diagram is used in place of a class diagram. Each component is a group of related functions in Contact_Book.py.

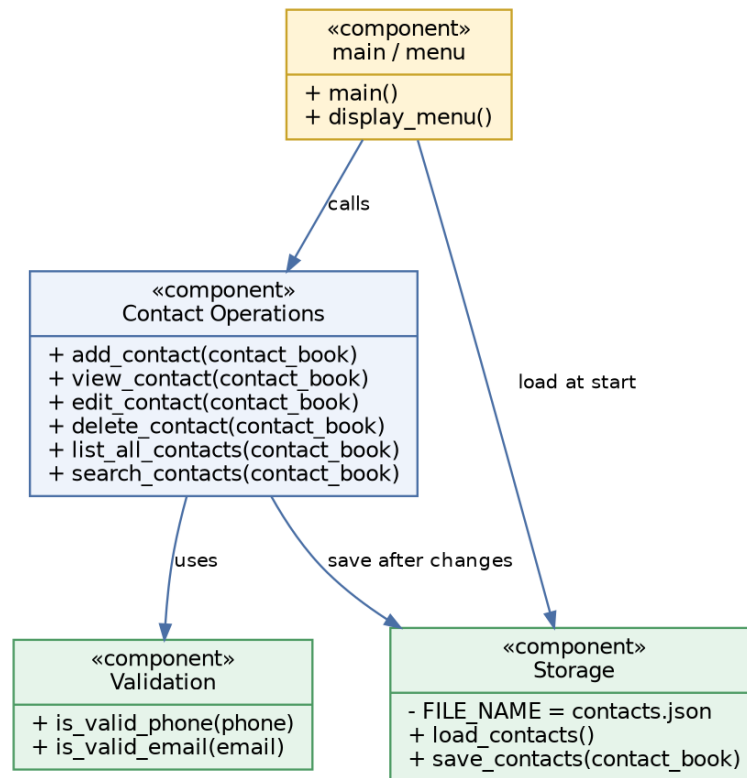


Figure 5: Component diagram

6.5 Data Model

The project uses a JSON file instead of a database, so there is no ER diagram with several tables. The data model below shows the single entity (Contact) and how it is stored in the file. The name is used as the key.



Figure 6: Data model of a contact and the storage file

Example of the file contents:

```

{
  "Ravi": {
    "phone": "9876543210",
    "email": "ravi@mail.com",
    "address": "Chennai"
  }
}

```

7. Design Decisions and Rationale

Decision	Reason
Dictionary keyed by name	Lookup, duplicate check and delete are all one step, and it maps directly to a JSON object.
JSON file for storage	Readable, built into Python, and enough for a small contact list. A database would be more than this project needs.
Functions in one file	The program is small, so one file with one function per task keeps it easy to follow.
Phone must be exactly 10 digits	Matches Indian mobile numbers and makes the rule simple to check and explain.
Email check is simple	Checking for "@" and a dot after it catches common typing mistakes without a complicated pattern.
Blank input keeps old value on edit	The user only has to type the field they want to change.
Save after every change	Nothing is lost if the program is closed suddenly.
Damaged file is renamed to .bak	If contacts.json cannot be read, it is kept as a backup so the next save does not overwrite it.
List is sorted by name	Output is always in the same order, which makes it easier to read.

8. Implementation Details

The program is a single file, `Contact_Book.py`, with the following functions.

Function	What it does
<code>display_menu()</code>	Prints the seven menu options.
<code>load_contacts()</code>	Reads <code>contacts.json</code> into a dictionary. Returns an empty dictionary if the file is missing, unreadable or damaged.
<code>save_contacts(contact_book)</code>	Writes the dictionary to <code>contacts.json</code> . Prints an error if writing fails.
<code>is_valid_phone(phone)</code>	True if the phone is made of digits only and has length 10.
<code>is_valid_email(email)</code>	True if the email has "@" and a dot after the "@".
<code>add_contact(contact_book)</code>	Reads details, validates them, stores the contact and saves.
<code>view_contact(contact_book)</code>	Shows one contact.
<code>edit_contact(contact_book)</code>	Updates the fields the user fills in and validates them.
<code>delete_contact(contact_book)</code>	Removes a contact and saves.
<code>list_all_contacts(contact_book)</code>	Prints every contact in alphabetical order.
<code>search_contacts(contact_book)</code>	Prints contacts whose name contains the search text.
<code>main()</code>	Loads the contacts and runs the menu loop until the user exits.

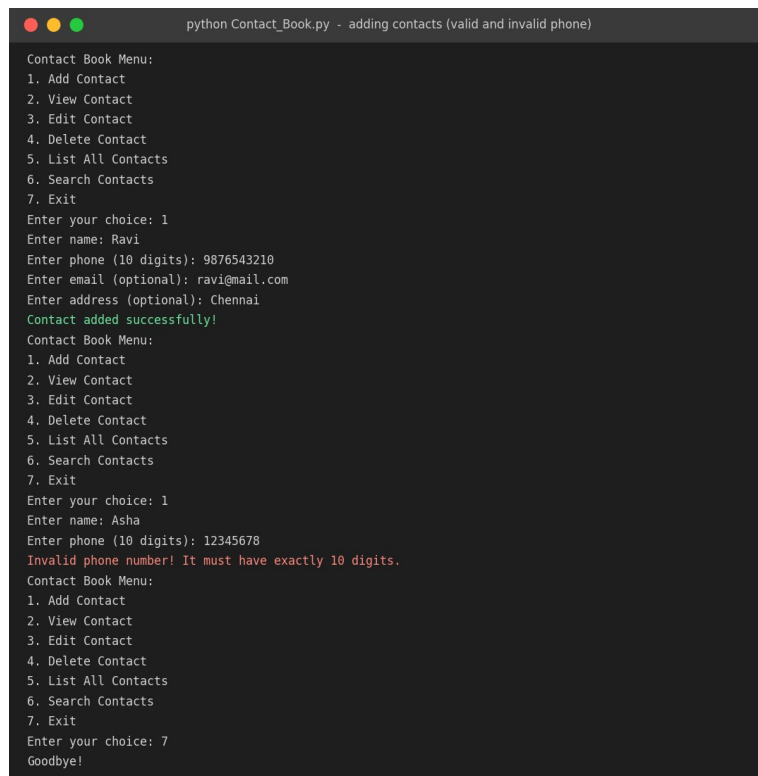
The save and load functions are the part that connects the program to the file:

```
def load_contacts():
    if not os.path.exists(FILE_NAME):
        return {}
    try:
        with open(FILE_NAME, "r") as f:
            return json.load(f)
    except json.JSONDecodeError:
        os.replace(FILE_NAME, FILE_NAME + ".bak")
        print("Contacts file was damaged. Starting fresh (old file saved as .bak).")
        return {}
    except OSError:
        print("Could not read the contacts file. Starting with an empty book.")
        return {}
```

The complete source code is submitted separately as a Word file and is also available in the GitHub repository.

9. Screenshots and Results

The screenshots below are the actual console output of the program for a few common cases.



```
python Contact_Book.py - adding contacts (valid and invalid phone)

Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 1
Enter name: Ravi
Enter phone (10 digits): 9876543210
Enter email (optional): ravi@mail.com
Enter address (optional): Chennai
Contact added successfully!
Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 1
Enter name: Asha
Enter phone (10 digits): 12345678
Invalid phone number! It must have exactly 10 digits.
Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 7
Goodbye!
```

Figure 7: Adding a valid contact and rejecting a short phone number

```
python Contact_Book.py - list all and search

Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 5
Name: anita
Phone: 9812345670
Email: anita@mail.com
Address: Vellore

Name: Ravi
Phone: 9876543210
Email: ravi@mail.com
Address: Chennai

Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 6
Enter name or part of a name to search: AN
Name: anita | Phone: 9812345670
Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 7
Goodbye!
```

Figure 8: Listing contacts in alphabetical order and searching for "AN"

```
python Contact_Book.py - edit and view a contact

Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 3
Enter name to edit: Ravi
Leave a field blank to keep the current value.
New phone: 9000000001
New email:
New address:
Contact updated successfully!
Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 2
Enter name to view: Ravi
Name: Ravi
Phone: 9000000001
Email: ravi@mail.com
Address: Chennai
Contact Book Menu:
1. Add Contact
2. View Contact
3. Edit Contact
4. Delete Contact
5. List All Contacts
6. Search Contacts
7. Exit
Enter your choice: 7
Goodbye!
```

Figure 9: Editing only the phone number and viewing the result

10. Testing Approach

The program was tested by running it with prepared inputs and comparing the output with the expected result. Every menu option was tested with valid input, invalid input and a missing contact. Two extra tests covered the file: one to check that contacts are still there after restarting, and one with a damaged contacts.json. Each test started from a known state, and all 19 tests passed.

ID	Test	Input	Expected result	Result
TC01	Add valid contact	Add: Ravi, 9876543210, ravi@mail.com, Chennai	Contact added successfully!	Pass
TC02	Add duplicate name	Add: Ravi again	Contact already exists!	Pass
TC03	Add with 8-digit phone	Add: phone 12345678	Invalid phone number message	Pass
TC04	Add with invalid email	Add: email asha.com	Invalid email address!	Pass
TC05	Add with empty name	Add: empty name	Name cannot be empty!	Pass
TC06	View existing contact	View: Ravi	Details of Ravi displayed	Pass
TC07	View missing contact	View: Nobody	Contact not found!	Pass
TC08	Edit phone only (blank fields kept)	Edit: Ravi, new phone only	Phone changed, email and address unchanged	Pass
TC09	Edit with invalid phone	Edit: Ravi, phone "abc"	Contact not updated	Pass
TC10	Edit missing contact	Edit: Nobody	Contact not found!	Pass
TC11	List all (alphabetical order)	List: three contacts added in a different order	anita, Ravi, Zoya in order	Pass
TC12	Search partial name (case-insensitive)	Search: "AN"	anita found	Pass
TC13	Search with no match	Search: "xyz"	No matching contacts found.	Pass
TC14	Persistence after restart	Close program, run again, view Zoya	Zoya still present in new run	Pass
TC15	Delete existing contact	Delete: Zoya, then view Zoya	Deleted, then not found	Pass
TC16	Delete missing contact	Delete: Nobody	Contact not found!	Pass

ID	Test	Input	Expected result	Result
TC17	Invalid menu choice	Menu choice: 9	Invalid choice message	Pass
TC18	List when book is empty	List with no contacts	No contacts available.	Pass
TC19	Corrupted contacts file	contacts.json contains {bad json	Warning shown, program continues, .bak created	Pass

11. Challenges Faced

- **Stopping bad data:** the first version accepted anything typed in. Deciding which rules were worth adding without making the program annoying to use took some thought. I kept them to a non-empty name, a 10-digit phone and a basic email check.
- **Edit with blank fields:** the edit function had to tell the difference between "keep the old value" and "the user typed something wrong". Blank now means keep, and anything typed is validated before the contact is changed.
- **Protecting saved data:** if contacts.json was damaged and the program simply started with an empty book, the next save would overwrite the old data. Renaming the damaged file to .bak solved this.
- **Testing a menu program:** it was slow to type the same inputs again and again, so I fed the inputs to the program from a script and checked the output.
- **Names as keys:** using the name as the dictionary key is simple, but it means two people with the same name cannot be stored, and "ravi" and "Ravi" count as different. I decided to accept this limit to keep the design simple.

12. Learnings and Key Takeaways

- Dictionaries are a good fit when data is looked up by a key.
- Small helper functions for validation keep the main functions short and easy to read.
- File handling needs error handling, because files can be missing or damaged.
- Checking input before changing any data avoids half-updated records.
- Writing test cases first, even a simple list, shows bugs and missing checks early.
- A clear prompt and message for every step makes a console program much easier to use.

13. Future Enhancements

- A graphical interface using Tkinter, or a web version.
- Import and export of contacts as CSV.
- More than one phone number per contact, and a unique ID instead of the name as key.

- Automatic unit tests using the unittest module.
- Password protection or encryption for the saved file.

14. References

- [1] Python Software Foundation. json: JSON encoder and decoder.
<https://docs.python.org/3/library/json.html>
- [2] Python Software Foundation. os: Miscellaneous operating system interfaces.
<https://docs.python.org/3/library/os.html>
- [3] Python Software Foundation. Built-in types: mapping types (dict).
<https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
- [4] Python Software Foundation. The Python Tutorial: Errors and exceptions.
<https://docs.python.org/3/tutorial/errors.html>
- [5] van Rossum, G., Warsaw, B., Coghlan, N. PEP 8: Style Guide for Python Code.
<https://peps.python.org/pep-0008/>
- [6] VITyarthi. Build Your Own Project: General Project Instructions and Submission Guidelines.
- [7] Graphviz. Graph visualization software, used to draw the diagrams. <https://graphviz.org>